

Software Source Code Registration

Company: Kapuletu Systems

Date Generated: August 26, 2026

CONFIDENTIAL AND PROPRIETARY

This document contains unpublished, proprietary source code of Kapuletu Systems. It is provided strictly for the purpose of copyright registration and may not be reproduced or distributed.

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

File: services/finance/ledger_service.py

```
import json
import hashlib
import logging
from common.utils import parse_uuid
from sqlalchemy.orm import Session, joinedload
from sqlalchemy import select, func

from models.transaction import Transaction
from models.campaign import Campaign
from services.finance.schemas import LedgerEntryOut, CampaignLedgerSummaryOut, LedgerResponse, IntegrityCheckOut
from services.audit.service import AuditService

logger = logging.getLogger(__name__)

class LedgerService:
    """
    LedgerService: Manages retrieval and cryptographic verification of finalized financial records.
    """
    def __init__(self, db: Session):
        self.db = db

    def _recalculate_hash(self, txn: Transaction) -> str:
        """
        Recalculates the SHA-256 integrity seal for a given transaction.
        Must perfectly match the generation logic in ApprovalService.
        """
        txn_payload = {
            "transaction_id": str(txn.transaction_id),
            "owner_id": str(txn.owner_id),
            "group_id": str(txn.group_id),
            "campaign_id": str(txn.campaign_id) if txn.campaign_id else None,
            "transaction_code": txn.transaction_code,
            "amount": float(txn.amount),
            "sender_phone": txn.sender_phone,
            "payment_method": txn.payment_method,
            "source_evidence": txn.source_evidence,
            "status": txn.status,
            "created_at": txn.created_at.isoformat()
        }
        serialized = json.dumps(txn_payload, sort_keys=True).encode()
        return hashlib.sha256(serialized).hexdigest()

    def verify_integrity(self, transaction_id: str, owner_id: str) -> IntegrityCheckOut:
        """
        Performs a deep cryptographic audit of a single transaction.
        """
        txn = self.db.execute(
            select(Transaction).where(
                Transaction.transaction_id == parse_uuid(transaction_id),
                Transaction.owner_id == parse_uuid(owner_id)
            )
        ).scalars().first()

        if not txn:
            raise ValueError("Transaction not found or access denied")

        recalculated = self._recalculate_hash(txn)
        is_valid = (recalculated == txn.ledger_hash)
        if not is_valid:
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        logger.critical(f"TAMPERING DETECTED: Transaction {transaction_id} hash mismatch!")
        AuditService(self.db).log_action(
            actor_id=owner_id,
            action="TAMPERING_DETECTED",
            entity_type="TRANSACTION",
            entity_id=str(transaction_id),
            details={"original_hash": txn.ledger_hash, "recalculated": recalculated}
        )

    return IntegrityCheckOut(
        transaction_id=str(txn.transaction_id),
        is_valid=is_valid,
        original_hash=txn.ledger_hash,
        recalculated_hash=recalculated,
        message="Valid immutable record" if is_valid else "TAMPERING DETECTED: Cryptographic seal broken."
    )

def get_campaign_ledger(self, campaign_id: str, owner_id: str) -> LedgerResponse:
    """
    Fetches the immutable ledger for a specific campaign, performing
    on-the-fly integrity verification.
    """
    # 1. Fetch Campaign Details
    campaign = self.db.execute(
        select(Campaign).where(Campaign.campaign_id == parse_uuid(campaign_id))
    ).scalars().first()

    # 2. Fetch all approved transactions for this campaign
    stmt = select(Transaction).options(joinedload(Transaction.allocations)).where(
        Transaction.owner_id == parse_uuid(owner_id),
        Transaction.campaign_id == parse_uuid(campaign_id),
        Transaction.status == "approved"
    ).order_by(Transaction.created_at.desc())

    txns = self.db.execute(stmt).scalars().unique().all()

    # 3. Process Entries and Verify Integrity
    entries = []
    total_raised = 0.0

    for txn in txns:
        # On-the-fly verification
        recalculated = self._recalculate_hash(txn)
        is_tampered = (recalculated != txn.ledger_hash)

        # If not tampered, add to trusted total
        if not is_tampered:
            total_raised += float(txn.amount)

    allocs = [{"member_name": a.member_name, "allocated_amount": float(a.allocated_amount)} for a in txns.allocations]

    entries.append(
        LedgerEntryOut(
            transaction_id=str(txn.transaction_id),
            owner_id=str(txn.owner_id),
            group_id=str(txn.group_id),
            campaign_id=str(txn.campaign_id) if txn.campaign_id else None,
            transaction_code=txn.transaction_code,
            amount=float(txn.amount),
            sender_phone=txn.sender_phone,
            sender_name=txn.sender_name,
            status=txn.status,

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```

        created_at=txn.created_at,
        ledger_hash=txn.ledger_hash,
        payment_method=txn.payment_method,
        source_evidence=txn.source_evidence,
        is_tampered=is_tampered,
        allocations=allocs
    )
)

# 4. Calculate Summary
summary = None
if campaign:
    target = float(campaign.target_amount or 0.0)
    progress = (total_raised / target * 100) if target > 0 else 0.0
    surplus = max(0.0, total_raised - target)
    summary = CampaignLedgerSummaryOut(
        campaign_id=str(campaign.campaign_id),
        title=campaign.title,
        target_amount=target,
        total_raised=total_raised,
        transaction_count=len(entries),
        progress_percentage=round(progress, 2),
        surplus_amount=surplus
    )

return LedgerResponse(summary=summary, entries=entries)

def get_global_ledger(self, owner_id: str) -> LedgerResponse:
    """
    Fetches the complete ledger for the owner.
    """
    stmt = select(Transaction).options(joinedload(Transaction.allocations)).where(
        Transaction.owner_id == parse_uuid(owner_id),
        Transaction.status == "approved"
    ).order_by(Transaction.created_at.desc())

    txns = self.db.execute(stmt).scalars().unique().all()

    entries = []
    for txn in txns:
        recalculated = self._recalculate_hash(txn)
        is_tampered = (recalculated != txn.ledger_hash)

        allocs = [{"member_name": a.member_name, "allocated_amount": float(a.allocated_amount)} for a in txn.allocations]

        entries.append(
            LedgerEntryOut(
                transaction_id=str(txn.transaction_id),
                owner_id=str(txn.owner_id),
                group_id=str(txn.group_id),
                campaign_id=str(txn.campaign_id) if txn.campaign_id else None,
                transaction_code=txn.transaction_code,
                amount=float(txn.amount),
                sender_phone=txn.sender_phone,
                sender_name=txn.sender_name,
                status=txn.status,
                created_at=txn.created_at,
                ledger_hash=txn.ledger_hash,
                payment_method=txn.payment_method,
                source_evidence=txn.source_evidence,
                is_tampered=is_tampered,
                allocations=allocs
            )
        )

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```
    )  
    )  
  
    return LedgerResponse(summary=None, entries=entries)
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

File: services/finance/ledger_router.py

```
from typing import List, Dict, Any
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from common.database import get_db
from common.auth_dependencies import get_verified_user
from services.finance.ledger_service import LedgerService
from services.finance.schemas import LedgerResponse, IntegrityCheckOut

router = APIRouter(prefix="/ledger", tags=["9. Ledger (Immutable)"])

@router.get("", response_model=LedgerResponse, summary="Get Global Ledger")
async def get_global_ledger(
    db: Session = Depends(get_db),
    current_user: Dict[str, Any] = Depends(get_verified_user)
):
    """
    Returns the complete, immutable ledger of all approved transactions.
    """
    service = LedgerService(db)
    return service.get_global_ledger(owner_id=current_user.get("sub"))

@router.get("/campaign/{campaign_id}", response_model=LedgerResponse, summary="Get Ledger by Campaign")
async def get_campaign_ledger(
    campaign_id: str,
    db: Session = Depends(get_db),
    current_user: Dict[str, Any] = Depends(get_verified_user)
):
    """
    Returns the immutable ledger for a specific campaign, including aggregates.
    """
    service = LedgerService(db)
    return service.get_campaign_ledger(campaign_id=campaign_id, owner_id=current_user.get("sub"))

@router.get("/verify/{transaction_id}", response_model=IntegrityCheckOut, summary="Cryptographic Audit")
async def verify_transaction_integrity(
    transaction_id: str,
    db: Session = Depends(get_db),
    current_user: Dict[str, Any] = Depends(get_verified_user)
):
    """
    Performs a deep cryptographic audit on a single transaction to ensure its data
    has not been tampered with since approval.
    """
    service = LedgerService(db)
    try:
        return service.verify_integrity(transaction_id=transaction_id, owner_id=current_user.get("sub"))
    except ValueError as e:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(e))
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

File: repositories/transaction_repo.py

```

from typing import List, Optional
from uuid import UUID

from sqlalchemy import select
from common.utils import parse_uuid
from sqlalchemy.orm import Session
import time

_UNAUTHORIZED_PHONE_CACHE = {}

from models.pending_transaction import PendingTransaction
from models.users import User

class TransactionRepository:
    """
    TransactionRepository: Encapsulates all database interactions related to transactions.

    This layer abstracts the SQLAlchemy logic away from the service layer,
    ensuring a clean separation of concerns and making the code easier to test.
    """

    def __init__(self, db: Session):
        """Initializes the repository with a database session."""
        self.db = db

    def insert_pending_transaction(self, pending_txn: PendingTransaction) -> PendingTransaction:
        """
        Persists a new pending transaction to the PostgreSQL database.

        Args:
            pending_txn (PendingTransaction): The model instance to save.

        Returns:
            PendingTransaction: The persisted instance with generated IDs and timestamps.
        """
        self.db.add(pending_txn)
        self.db.commit()
        self.db.refresh(pending_txn)
        return pending_txn

    def check_duplicate_transaction_code(self, transaction_code: str, owner_id: UUID) -> bool:
        """
        Checks if a transaction code (or idempotency hash) already exists in
        EITHER the pending queue or the finalized ledger FOR THIS SPECIFIC USER.

        Args:
            transaction_code (str): The unique code to check.
            owner_id (UUID): The user to check against.

        Returns:
            bool: True if a match is found in either table, False otherwise.
        """
        if not transaction_code:
            return False

        # 1. Check Pending Transactions
        from models.transaction import Transaction
        pending_exists = self.db.query(PendingTransaction).filter(

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        PendingTransaction.transaction_code == transaction_code,
        PendingTransaction.owner_id == parse_uuid(owner_id)
    ).first() is not None

    if pending_exists:
        return True

    # 2. Check Finalized Transactions
    finalized_exists = self.db.query(Transaction).filter(
        Transaction.transaction_code == transaction_code,
        Transaction.owner_id == parse_uuid(owner_id)
    ).first() is not None

    return finalized_exists

def resolve_owner_by_phone(self, phone_number: str) -> Optional[User]:
    """
    Identifies the Treasurer (User) associated with a specific phone number.
    This is used during ingestion to route messages to the correct account.
    Includes a 5-minute TTL cache for unauthorized numbers to prevent DB DoS attacks.

    Args:
        phone_number (str): The sender's phone number.

    Returns:
        Optional[User]: The User record if found, else None.
    """
    now = time.time()

    # Check Circuit Breaker / Cache
    if phone_number in _UNAUTHORIZED_PHONE_CACHE:
        if now - _UNAUTHORIZED_PHONE_CACHE[phone_number] < 300: # 5 minute TTL
            return None
        else:
            del _UNAUTHORIZED_PHONE_CACHE[phone_number]

    stmt = select(User).where(User.phone_number == phone_number)
    user = self.db.execute(stmt).scalars().first()

    if not user:
        _UNAUTHORIZED_PHONE_CACHE[phone_number] = now
        # Cap memory growth
        if len(_UNAUTHORIZED_PHONE_CACHE) > 10000:
            _UNAUTHORIZED_PHONE_CACHE.clear()

    return user

def fetch_pending_transactions_by_owner(self, owner_id: UUID, skip: int = 0, limit: int = 100, search: Optional[str] = None, filter_val: Optional[str] = None, status: str = "all"):
    """
    Retrieves transactions for a specific treasurer with pagination and status filtering.

    Args:
        owner_id (UUID): The unique ID of the treasurer.
        skip (int): Pagination offset.
        limit (int): Pagination limit.
        search (Optional[str]): Search query for name or code.
        filter_val (Optional[str]): Date filter.
        status (str): "pending", "approved", "rejected", or "all".

    Returns:
        tuple: (List[PendingTransaction], total_count)
    """

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```

from sqlalchemy import func, or_
from datetime import datetime, timedelta
from models.group import Group
from models.campaign import Campaign

base_query = self.db.query(PendingTransaction, Group.group_name, Campaign.title, Group.slug.label("group_slug"),
                           Group, PendingTransaction.group_id == Group.group_id
).outerjoin(
    Campaign, PendingTransaction.campaign_id == Campaign.campaign_id
).filter(
    PendingTransaction.owner_id == parse_uuid(owner_id)
)

if status == "pending":
    base_query = base_query.filter(PendingTransaction.is_processed == False)
elif status == "approved":
    base_query = base_query.filter(PendingTransaction.is_processed == True, PendingTransaction.workflow_id == workflow_id)
elif status == "rejected":
    base_query = base_query.filter(PendingTransaction.is_processed == True, PendingTransaction.workflow_id == workflow_id)

if search:
    search_term = f"%{search}%"
    base_query = base_query.filter(or_(
        PendingTransaction.sender_name.ilike(search_term),
        PendingTransaction.transaction_code.ilike(search_term)
    ))

if filter_val:
    now = datetime.utcnow()
    if filter_val == "today":
        start_date = now.replace(hour=0, minute=0, second=0, microsecond=0)
        base_query = base_query.filter(PendingTransaction.created_at >= start_date)
    elif filter_val == "this_week":
        start_date = now - timedelta(days=now.weekday())
        start_date = start_date.replace(hour=0, minute=0, second=0, microsecond=0)
        base_query = base_query.filter(PendingTransaction.created_at >= start_date)
    elif filter_val == "this_month":
        start_date = now.replace(day=1, hour=0, minute=0, second=0, microsecond=0)
        base_query = base_query.filter(PendingTransaction.created_at >= start_date)
    elif filter_val == "this_year":
        start_date = now.replace(month=1, day=1, hour=0, minute=0, second=0, microsecond=0)
        base_query = base_query.filter(PendingTransaction.created_at >= start_date)

total = base_query.count()

if sort_by == "amount":
    if sort_order == "asc":
        base_query = base_query.order_by(PendingTransaction.amount.asc(), PendingTransaction.created_at.desc())
    else:
        base_query = base_query.order_by(PendingTransaction.amount.desc(), PendingTransaction.created_at.desc())
else:
    if sort_order == "asc":
        base_query = base_query.order_by(PendingTransaction.created_at.asc())
    else:
        base_query = base_query.order_by(PendingTransaction.created_at.desc())

results = base_query.offset(skip).limit(limit).all()

items = []
for pending, group_name, campaign_title, group_slug, campaign_slug in results:
    pending.assigned_group_name = group_name
    pending.assigned_group_slug = group_slug

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        pending.assigned_campaign_name = campaign_title
        pending.assigned_campaign_slug = campaign_slug
        items.append(pending)

    return items, total

def fetch_pending_transactions_by_campaign(self, campaign_id: UUID, owner_id: UUID, skip: int = 0, limit: int = 100):
    """Fetches pending transactions for a specific campaign."""
    from sqlalchemy import func
    count_stmt = select(func.count()).select_from(PendingTransaction).where(
        PendingTransaction.owner_id == parse_uuid(owner_id),
        PendingTransaction.campaign_id == parse_uuid(campaign_id),
        PendingTransaction.is_processed == False
    )
    total = self.db.execute(count_stmt).scalar()

    stmt = select(PendingTransaction).where(
        PendingTransaction.owner_id == parse_uuid(owner_id),
        PendingTransaction.campaign_id == parse_uuid(campaign_id),
        PendingTransaction.is_processed == False
    )

    if sort_by == "amount":
        if sort_order == "asc":
            stmt = stmt.order_by(PendingTransaction.amount.asc(), PendingTransaction.created_at.desc())
        else:
            stmt = stmt.order_by(PendingTransaction.amount.desc(), PendingTransaction.created_at.desc())
    else:
        if sort_order == "asc":
            stmt = stmt.order_by(PendingTransaction.created_at.asc())
        else:
            stmt = stmt.order_by(PendingTransaction.created_at.desc())

    stmt = stmt.offset(skip).limit(limit)

    items = self.db.execute(stmt).scalars().all()
    return items, total

def fetch_pending_transaction_by_id(self, pending_id: str, owner_id: UUID) -> Optional[PendingTransaction]:
    """
    Retrieves a specific pending transaction for a specific treasurer.

    Args:
        pending_id (str): The unique ID of the pending transaction.
        owner_id (UUID): The unique ID of the treasurer.

    Returns:
        Optional[PendingTransaction]: The pending transaction if found and owned by user, else None.
    """
    stmt = select(PendingTransaction).where(
        PendingTransaction.pending_id == parse_uuid(pending_id),
        PendingTransaction.owner_id == parse_uuid(owner_id),
        PendingTransaction.is_processed == False
    )
    return self.db.execute(stmt).scalars().first()

def fetch_inbox_history(
    self, owner_id: UUID, skip: int = 0, limit: int = 100,
    status: Optional[str] = None, search: Optional[str] = None,
    date_from: Optional[str] = None, date_to: Optional[str] = None,
    sort_by: Optional[str] = "date", sort_order: Optional[str] = "desc"
):
```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```

"""
Retrieves processed inbox transactions for history.
"""
from sqlalchemy import or_, desc
from datetime import datetime

query = self.db.query(
    PendingTransaction,
    User.first_name,
    User.last_name
).outerjoin(User, PendingTransaction.processed_by == User.user_id).filter(
    PendingTransaction.owner_id == parse_uuid(owner_id),
    PendingTransaction.is_processed == True
)

if status and status != "all":
    query = query.filter(PendingTransaction.workflow_status == status)

if search:
    search_term = f"%{search}%"
    query = query.filter(or_(
        PendingTransaction.sender_name.ilike(search_term),
        PendingTransaction.transaction_code.ilike(search_term)
    ))

if date_from:
    try:
        dt_from = datetime.fromisoformat(date_from.replace('Z', '+00:00'))
        query = query.filter(PendingTransaction.processed_at >= dt_from)
    except ValueError:
        pass

if date_to:
    try:
        dt_to = datetime.fromisoformat(date_to.replace('Z', '+00:00'))
        query = query.filter(PendingTransaction.processed_at <= dt_to)
    except ValueError:
        pass

total = query.count()

if sort_by == "amount":
    if sort_order == "asc":
        query = query.order_by(PendingTransaction.amount.asc(), desc(PendingTransaction.processed_at))
    else:
        query = query.order_by(PendingTransaction.amount.desc(), desc(PendingTransaction.processed_at))
else:
    if sort_order == "asc":
        query = query.order_by(PendingTransaction.processed_at.asc())
    else:
        query = query.order_by(desc(PendingTransaction.processed_at))

items = query.offset(skip).limit(limit).all()

results = []
for pending, fname, lname in items:
    processed_by_name = f"{fname or ''} {lname or ''}".strip() if fname or lname else None
    results.append({
        "pending": pending,
        "processed_by_name": processed_by_name
    })
return results, total

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

File: services/ingestion/parser_engine.py

```
import logging
import os
import re
from typing import Any, Dict

import spacy

logger = logging.getLogger(__name__)

# THE AI CORE: Loads the proprietary trained weights if available
MODEL_PATH = os.path.join(os.path.dirname(__file__), "../../models/kapuletu_ai_v1")

class ModelBasedParser:
    """
    Kapuletu AI Core: Intelligent Financial Message Parsing Engine.

    This engine combines Machine Learning (SpaCy NER) with deterministic heuristics
    to extract structured financial data from unstructured text messages (WhatsApp/SMS).

    The architecture follows a 'Safety-First' approach:
    1. Named Entity Recognition (NER) identifies primary fields.
    2. Heuristics (Regex) act as a fallback/validation layer.
    3. Cleaning logic normalizes financial strings into floats/standard codes.
    """

    def __init__(self):
        """
        Initializes the NLP pipeline.
        Attempts to load a custom-trained SpaCy model for higher accuracy.
        Falls back to a blank English model if the proprietary weights are missing.
        """
        try:
            if os.path.exists(MODEL_PATH):
                logger.info(f"AI Core: Loading production model weights from {MODEL_PATH}")
                self.nlp = spacy.load(MODEL_PATH)
                self.is_trained = True
            else:
                logger.warning("AI Core: Custom model weights not found. Falling back to heuristic-heavy mode.")
                self.nlp = spacy.blank("en") # Efficient but requires fallback regex
                self.is_trained = False
        except Exception as e:
            logger.error(f"AI Core Initialization Error: {e}")
            self.nlp = spacy.blank("en")
            self.is_trained = False

    def parse(self, message_text: str) -> Dict[str, Any]:
        """
        Executes the full parsing pipeline on a single message string.

        Logic Flow:
        1. Contextual Extraction: SpaCy identifies entities like SENDER, AMOUNT, PURPOSE.
        2. Safeguard Check: If critical data (amount/name) is missing, regex logic is triggered.
        3. Confidence Scoring: Assigns a score based on whether the custom model was used.

        Args:
            message_text (str): The raw text of the received message.

        Returns:
            dict: Structured data containing amount, transaction_code, sender_name, purpose.
        """
```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```

"""
# Run the NLP pipeline with error handling
try:
    doc = self.nlp(message_text)
except Exception as e:
    logger.error(f"AI Core Runtime Error: {e}. Falling back to heuristics.")
    # Create a blank doc if model fails
    doc = spacy.blank("en")(message_text)
    self.is_trained = False

# Initialize the data schema
data = {
    "sender_name": None,
    "amount": 0.0,
    "transaction_code": None,
    "purpose": None,
    "provider": None,
    "transaction_date": None,
    "account": None,
    "phone": None,
    "confidence_score": 0.0
}

# 1. AI-Driven Extraction using Named Entities
# We iterate through the identified 'entities' in the text.
for ent in doc.ents:
    if ent.label_ == "SENDER":
        data["sender_name"] = ent.text
    elif ent.label_ == "AMOUNT":
        data["amount"] = self._clean_amount(ent.text)
    elif ent.label_ == "CODE":
        data["transaction_code"] = ent.text
    elif ent.label_ == "PURPOSE":
        data["purpose"] = ent.text
    elif ent.label_ == "PROVIDER":
        data["provider"] = ent.text
    elif ent.label_ == "DATE":
        data["transaction_date"] = ent.text
    elif ent.label_ == "ACCOUNT":
        data["account"] = ent.text
    elif ent.label_ == "PHONE":
        data["phone"] = self._normalize_phone(ent.text)

# 2. Heuristic Safeguard (Regex Fallback)
# If the AI model missed critical fields (common with unstructured SMS),
# we run deterministic regex patterns to 'catch' the data.
if not data["amount"] or not data["sender_name"] or not data["transaction_code"]:
    data = self._safeguard_heuristics(message_text, data)

# Unconditionally check for phone number if not found by AI
if not data["phone"]:
    data = self._extract_phone_heuristic(message_text, data)

# 3. Confidence Calculation
# In production, this would be based on the model's 'prob' output.
# Here we use a static indicator of model presence.
data["confidence_score"] = 1.0 if self.is_trained else 0.7
return data

def _safeguard_heuristics(self, text: str, existing_data: Dict[str, Any]) -> Dict[str, Any]:
    """
    Regex-based fallback logic to ensure high capture rates.

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

    Uses common Kenyan financial message patterns (Ksh, MPESA codes).
    """
    # Capture Amount (e.g. 'Ksh 1,500.00' or 'Ksh1500')
    if not existing_data["amount"]:
        amt_match = re.search(r"Ksh\s?([\d,.]+)", text, re.I)
        if amt_match:
            existing_data["amount"] = self._clean_amount(amt_match.group(1))

    # Capture Transaction Code (Alpha-numeric, usually 10+ chars)
    if not existing_data["transaction_code"]:
        code_match = re.search(r"\b([A-Z\d]{10,})\b", text)
        if code_match:
            existing_data["transaction_code"] = code_match.group(1)

    return existing_data

def _clean_amount(self, amt_str: str) -> float:
    """
    Normalizes currency strings into machine-readable floats.
    Removes currency symbols, commas, and whitespace.
    """
    try:
        import re
        clean_str = re.sub(r"^\d.", "", amt_str)
        # Remove trailing/multiple dots if they appear by accident
        if clean_str.count(".") > 1:
            parts = clean_str.split(".")
            clean_str = "".join(parts[:-1]) + "." + parts[-1]
        return float(clean_str)
    except (ValueError, TypeError):
        return 0.0

def _extract_phone_heuristic(self, text: str, existing_data: Dict[str, Any]) -> Dict[str, Any]:
    """
    Extracts phone numbers using Kenyan mobile number formats (e.g. 07xx, 2547xx).
    """
    # Look for 9 to 12 digit numbers matching Safaricom, Airtel, Telkom patterns
    phone_match = re.search(r"\b(?:07|01|2547|2541|\+2547|\+2541)\d{8}\b", text)
    if phone_match:
        existing_data["phone"] = self._normalize_phone(phone_match.group(0))
    return existing_data

def _normalize_phone(self, phone_str: str) -> str:
    """
    Normalizes any valid Kenyan phone format to international E.164 (+254...)
    """
    clean_phone = re.sub(r"^\d+", "", phone_str)
    if clean_phone.startswith("07") or clean_phone.startswith("01"):
        return "+254" + clean_phone[1:]
    elif clean_phone.startswith("254"):
        return "+" + clean_phone
    elif clean_phone.startswith("+254"):
        return clean_phone
    return phone_str

_parser_instance = None

def parse_message(message_text: str) -> Dict[str, Any]:
    """Utility wrapper for one-off parsing calls using a cached singleton."""
    global _parser_instance
    if _parser_instance is None:
        _parser_instance = ModelBasedParser()

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```
return _parser_instance.parse(message_text)
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

File: services/ingestion/handler.py

```

from common.utils import parse_uuid
import json
import logging
import os
import requests

from common.database import SessionLocal
from services.ingestion.service import IngestionService
from common.config import get_config

# Setup logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def handler(event, context):
    """
    Primary entry point for the Meta WhatsApp Cloud API Webhook.
    """
    logger.info("Received Ingestion Webhook request")

    http_method = event.get("httpMethod", "POST")
    query_params = event.get("queryStringParameters") or {}

    config = get_config()

    # 1. Meta Webhook Verification Challenge (GET)
    if http_method == "GET":
        mode = query_params.get("hub.mode")
        token = query_params.get("hub.verify_token")
        challenge = query_params.get("hub.challenge")

        if mode == "subscribe" and token == config.META_VERIFY_TOKEN:
            logger.info("Meta Webhook Verification successful.")
            return {
                "statusCode": 200,
                "headers": {"Content-Type": "text/plain"},
                "body": str(challenge)
            }
        else:
            logger.warning("Meta Webhook Verification failed: Invalid token.")
            return {"statusCode": 403, "body": "Verification failed"}

    # 2. Payload Extraction & Normalization (POST)
    body_str = event.get("body", "{}")
    is_base64 = event.get("isBase64Encoded", False)

    if is_base64:
        import base64
        try:
            body_str = base64.b64decode(body_str).decode("utf-8")
        except Exception as e:
            logger.error(f"Failed to decode base64 body: {e}")
            return {"statusCode": 400, "body": json.dumps({"error": "invalid_encoding"})}

    # 3. SQS Decoupling (If configured)
    sqs_queue_url = os.environ.get("SQS_QUEUE_URL")
    if sqs_queue_url:
        import boto3
        sqs = boto3.client("sqs", region_name=os.environ.get("AWS_REGION", "eu-west-1"))

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```

try:
    sqs.send_message(QueueUrl=sqs_queue_url, MessageBody=body_str)
    logger.info("Successfully queued webhook payload to SQS.")
    return {"statusCode": 200, "body": "OK"}
except Exception as e:
    logger.error(f"Failed to queue to SQS: {e}")
    # Fall back to synchronous processing if SQS fails

# 4. Asynchronous Thread Fallback (Local Dev or SQS Bypassed)
import threading

def background_task(body, conf):
    try:
        process_ingestion(body, conf)
    except Exception as e:
        logger.error(f"Background thread processing failed: {e}")

thread = threading.Thread(target=background_task, args=(body_str, config))
thread.start()

# Always return 200 OK instantly to Meta to prevent retry floods
return {"statusCode": 200, "body": "OK"}

def process_sqs_record(record):
    """
    Entry point for SQS worker triggering from the Webhook Queue.
    """
    logger.info("Processing SQS Webhook Record")
    body_str = record.get("body", "{}")
    config = get_config()
    return process_ingestion(body_str, config)

def process_ingestion(body_str: str, config):
    """
    Core logic: parses payload, saves to database, and sends WhatsApp reply.
    """
    try:
        payload_data = json.loads(body_str)
    except json.JSONDecodeError:
        return {"statusCode": 400, "body": "Invalid JSON"}

    # Meta webhook payloads are deeply nested
    # entry -> changes -> value
    try:
        entry = payload_data.get("entry", [])[0]
        changes = entry.get("changes", [])[0]
        value = changes.get("value", {})
    except IndexError:
        return {"statusCode": 200, "body": "OK"} # Acknowledge malformed to prevent retries

    # Check if this is a status update (delivery/read receipt)
    if "statuses" in value:
        status_info = value["statuses"][0]
        logger.info(f"Received status update: {status_info.get('status')} for message ID {status_info.get('id')}")
        # We don't process these with the AI, just acknowledge
        return {"statusCode": 200, "body": "OK"}

    # Check if this is a message
    if "messages" not in value:
        return {"statusCode": 200, "body": "OK"} # Acknowledge other types of webhooks

    message_info = value["messages"][0]

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```
# We care about text and interactive messages
msg_type = message_info.get("type")
if msg_type not in ["text", "interactive"]:
    logger.info(f"Ignoring message type: {msg_type}")
    return {"statusCode": 200, "body": "OK"}

# Extract required fields
sender_phone = message_info.get("from")
if not sender_phone:
    return {"statusCode": 400, "body": "Missing sender phone"}

# Meta sends phone numbers without the '+' sign (e.g., 254712345678).
if not sender_phone.startswith("+"):
    sender_phone = f"+{sender_phone}"

message_body = ""
interactive_id = None

if msg_type == "text":
    message_body = message_info.get("text", {}).get("body", "")
elif msg_type == "interactive":
    interactive = message_info.get("interactive", {})
    if interactive.get("type") == "list_reply":
        interactive_id = interactive.get("list_reply", {}).get("id")
        message_body = interactive.get("list_reply", {}).get("title", "")

if not message_body and not interactive_id:
    return {"statusCode": 400, "body": "Missing message content"}

# Normalize payload for the existing service architecture
normalized_payload = {
    "From": sender_phone,
    "Body": message_body
}

# 3. Database Session Initialization
db = SessionLocal()

try:
    from models.whatsapp_blocklist import WhatsAppBlocklist
    import datetime

    # 3.0 Check if user is blocked (gracefully handles missing table during migration window)
    try:
        blocked_record = db.query(WhatsAppBlocklist).filter(WhatsAppBlocklist.phone_number == sender_phone)
        if blocked_record and blocked_record.is_blocked:
            # Auto-unblock if last attempt was more than 24 hours ago
            now_utc = datetime.datetime.now(datetime.timezone.utc)
            last_attempt = blocked_record.last_attempt_at
            if last_attempt and last_attempt.tzinfo is None:
                last_attempt = last_attempt.replace(tzinfo=datetime.timezone.utc)

            if last_attempt and (now_utc - last_attempt).total_seconds() > 86400:
                blocked_record.is_blocked = False
                blocked_record.attempt_count = 0
                db.commit()
            else:
                # Still blocked, silently drop the request
                return {"statusCode": 200, "body": "OK"}
    except Exception as _blocklist_err:
        logger.warning(f"Blocklist table not available yet, skipping check: {_blocklist_err}")
        db.rollback()
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```
# --- Interactive Report Flow ---
from repositories.transaction_repo import TransactionRepository
from models.campaign import Campaign
from models.group import Group
from services.reporting.daily_summary import generate_campaign_whatsapp_report

# 3.1 Check if it's an interactive menu selection for a report or approval
if interactive_id:
    if interactive_id == "GREETING_WORKSPACE":
        send_meta_reply(sender_phone, f"You can access your KapuLetu dashboard here: {config.FRONTEND_URL}")
        return {"statusCode": 200, "body": "OK"}
    elif interactive_id == "GREETING_APPROVALS":
        send_meta_reply(sender_phone, f"To review and approve pending member transactions, please visit {config.FRONTEND_URL}")
        return {"statusCode": 200, "body": "OK"}
    elif interactive_id == "GREETING_HELP":
        send_whatsapp_help_menu(sender_phone, config)
        return {"statusCode": 200, "body": "OK"}
    elif interactive_id == "HELP_GROUP":
        send_meta_reply(sender_phone, "To create a new savings group, log in to your online workspace at {config.FRONTEND_URL}")
        return {"statusCode": 200, "body": "OK"}
    elif interactive_id == "HELP_CAMPAIGN":
        send_meta_reply(sender_phone, "Once a group is active, you can create a campaign directly via WhatsApp")
        return {"statusCode": 200, "body": "OK"}
    elif interactive_id == "HELP_APPROVE":
        send_meta_reply(sender_phone, "When members send payments, you will receive a notification. You can toggle this in the settings")
        return {"statusCode": 200, "body": "OK"}
    elif interactive_id == "HELP_BOT":
        send_meta_reply(sender_phone, "You can text the bot at any time using commands like 'REPORT' to generate a report")
        return {"statusCode": 200, "body": "OK"}
    elif interactive_id == "HELP_SETTINGS":
        send_meta_reply(sender_phone, "To customize your report formats, toggle WhatsApp features, or manage your profile, visit {config.FRONTEND_URL}")
        return {"statusCode": 200, "body": "OK"}
    if interactive_id and interactive_id.startswith("REPORT_CAMPAIGN_"):
        campaign_id = interactive_id.replace("REPORT_CAMPAIGN_", "")
        try:
            report_text = generate_campaign_whatsapp_report(db, campaign_id)
            dispatch_whatsapp_report(db, sender_phone, campaign_id, report_text, config)

            from repositories.transaction_repo import TransactionRepository
            repo = TransactionRepository(db)
            owner = repo.resolve_owner_by_phone(sender_phone)
            if owner:
                from services.audit.service import AuditService
                # Ensure we have the campaign title
                campaign_title = "Campaign"
                campaign_record = db.query(Campaign).filter(Campaign.campaign_id == parse_uuid(campaign_id))
                if campaign_record:
                    campaign_title = campaign_record.title

                AuditService(db).log_action(
                    actor_id=str(owner.user_id),
                    action="REPORT_GENERATED",
                    entity_type="campaign",
                    entity_id=campaign_id,
                    details={
                        "message": f"WhatsApp report generated for \"{campaign_title}\"",
                        "campaign_id": campaign_id
                    }
                )
        except Exception as e:
            logger.error(f"Failed to generate report for campaign {campaign_id}: {e}")
            send_meta_reply(sender_phone, "Error generating report. Please try again.", config)
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        return {"statusCode": 200, "body": "OK"}

    if interactive_id and interactive_id.startswith("APPROVE_"):
        # Format: APPROVE_{pending_id}_{campaign_id}
        parts = interactive_id.split("_")
        if len(parts) >= 3:
            pending_id = parts[1]
            campaign_id = parts[2]

            from services.approval.service import ApprovalService
            from models.campaign import Campaign
            from models.pending_transaction import PendingTransaction

            campaign = db.query(Campaign).filter(Campaign.campaign_id == parse_uuid(campaign_id)).first()
            if not campaign:
                send_meta_reply(sender_phone, "Error: Campaign not found.", config)
                return {"statusCode": 200, "body": "OK"}

            pending = db.query(PendingTransaction).filter(PendingTransaction.pending_id == parse_uuid(pending_id)).first()
            if not pending:
                send_meta_reply(sender_phone, "Error: Transaction not found.", config)
                return {"statusCode": 200, "body": "OK"}

            approval_service = ApprovalService(db)
            try:
                approval_service.approve_transaction(
                    pending_txn_id=pending_id,
                    treasurer_id=str(campaign.group.owner_id),
                    group_id=str(campaign.group_id),
                    campaign_id=campaign_id
                )
                send_meta_reply(sender_phone, "Success! The transaction has been approved and recorded in the system.", config)
            except Exception as e:
                logger.error(f"Failed to approve transaction {pending_id}: {e}")
                send_meta_reply(sender_phone, f"Error approving transaction: {e}", config)
                send_meta_reply(sender_phone, f"Error approving transaction: {e}", config)
            return {"statusCode": 200, "body": "OK"}

    if interactive_id and interactive_id.startswith("NEW_CAMP_"):
        # Format: NEW_CAMP_{group_id}_{base64_encoded_title}
        parts = interactive_id.split("_", 3)
        if len(parts) >= 4:
            group_id = parts[2]
            import base64
            try:
                title = base64.b64decode(parts[3]).decode('utf-8')
            except Exception:
                title = "New Campaign"

            from models.group import Group
            from models.campaign import Campaign
            import uuid

            group = db.query(Group).filter(Group.group_id == parse_uuid(group_id)).first()
            if not group:
                send_meta_reply(sender_phone, "Error: Group not found.", config)
                return {"statusCode": 200, "body": "OK"}

            new_camp = Campaign(
                campaign_id=uuid.uuid4(),
                group_id=group.group_id,
                title=title,

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        status="active",
        is_active=True
    )
    db.add(new_camp)
    db.commit()

    from services.audit.service import AuditService
    AuditService(db).log_action(
        actor_id=str(group.owner_id),
        action="CAMPAIGN_CREATED",
        entity_type="campaign",
        entity_id=str(new_camp.campaign_id),
        details={
            "message": f"New campaign \"{title}\" created via WhatsApp",
            "campaign_id": str(new_camp.campaign_id),
            "group_id": str(group.group_id),
            "group_slug": group.slug,
            "campaign_slug": new_camp.slug
        }
    )
    send_meta_reply(sender_phone, f"Success! Campaign '{title}' created under group '{group.group_name}'")
    return {"statusCode": 200, "body": "OK"}

# 3.2 Check if the user is explicitly requesting a text command
if msg_type == "text":
    repo = TransactionRepository(db)
    owner = repo.resolve_owner_by_phone(sender_phone)

    message_upper = message_body.strip().upper()
    greetings = ["HI", "HELLO", "HOLLA", "HOLA", "HEY", "GOOD MORNING", "GOOD AFTERNOON", "GOOD EVENING"]

    if message_upper in greetings:
        if not owner:
            handle_unauthorized_access(sender_phone, config, db)
            return {"statusCode": 200, "body": "OK"}
        send_whatsapp_main_menu(sender_phone, config)
        return {"statusCode": 200, "body": "OK"}

    if message_upper == "HELP":
        if not owner:
            handle_unauthorized_access(sender_phone, config, db)
            return {"statusCode": 200, "body": "OK"}
        send_whatsapp_help_menu(sender_phone, config)
        return {"statusCode": 200, "body": "OK"}

    if message_upper == "REPORT":
        if not owner:
            handle_unauthorized_access(sender_phone, config, db)
            return {"statusCode": 200, "body": "OK"}

    from services.settings.settings_service import SettingsService
    settings_service = SettingsService(db)
    user_settings = settings_service.get_global_settings(str(owner.user_id))

    if not user_settings.reporting.allow_whatsapp_reports:
        send_meta_reply(sender_phone, "WhatsApp reporting is currently disabled. Please enable it in settings")
        return {"statusCode": 200, "body": "OK"}

    # Query all active campaigns for the user
    active_campaigns = db.query(Campaign).join(Group).filter(
        Group.owner_id == parse_uuid(owner.user_id),
        Campaign.is_active == True
    )

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

).order_by(Campaign.created_at.desc()).all()

if len(active_campaigns) == 0:
    send_meta_reply(sender_phone, "Notice: You do not have any active campaigns.", config)
elif len(active_campaigns) == 1:
    # Only one campaign, send it directly
    report_text = generate_campaign_whatsapp_report(db, str(active_campaigns[0].campaign_id))
    dispatch_whatsapp_report(db, sender_phone, str(active_campaigns[0].campaign_id), report_text)

    from services.audit.service import AuditService
    AuditService(db).log_action(
        actor_id=str(owner.user_id),
        action="REPORT_GENERATED",
        entity_type="campaign",
        entity_id=str(active_campaigns[0].campaign_id),
        details={
            "message": f"WhatsApp report generated for \"{active_campaigns[0].title}\"",
            "campaign_id": str(active_campaigns[0].campaign_id)
        }
    )
else:
    # Multiple campaigns, construct an interactive list
    rows = []
    for c in active_campaigns[:10]: # WhatsApp limits to 10 rows per list
        # Title is limited to 24 chars
        title = c.title[:24]
        rows.append({
            "id": f"REPORT_CAMPAIGN_{c.campaign_id}",
            "title": title
        })

    sections = [{
        "title": "Active Campaigns",
        "rows": rows
    }]

    body_text = "You have multiple active campaigns. Please select the campaign you want to generate a report for."
    send_meta_interactive_list(sender_phone, body_text, "Select Campaign", sections, config)

    return {"statusCode": 200, "body": "OK"}

# 3.3 Check conversational NLP intents (KapuletU AI)
if msg_type == "text":
    from services.ingestion.local_intent_parser import intent_parser
    from repositories.transaction_repo import TransactionRepository
    from services.settings.settings_service import SettingsService

    repo = TransactionRepository(db)
    owner = repo.resolve_owner_by_phone(sender_phone)

    if owner:
        settings_service = SettingsService(db)
        user_settings = settings_service.get_global_settings(str(owner.user_id))

        parsed_intent = intent_parser.detect_intent(message_body)

        if parsed_intent["intent"] in ["create_group", "create_campaign"]:
            if not user_settings.automation.allow_whatsapp_creation:
                send_meta_reply(sender_phone, "WhatsApp creation commands are currently disabled. Please contact support for more information.")
                return {"statusCode": 200, "body": "OK"}

            if parsed_intent["intent"] == "create_group":

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```

group_name = parsed_intent["entities"].get("group_name")
if not group_name:
    send_meta_reply(sender_phone, "I see you want to create a group! To do this, please
    return {"statusCode": 200, "body": "OK"}

from models.group import Group
import uuid
import datetime

# Check daily limit: Max 2 groups per day
yesterday = datetime.datetime.utcnow() - datetime.timedelta(days=1)
daily_group_count = db.query(Group).filter(
    Group.owner_id == owner.user_id,
    Group.created_at >= yesterday
).count()

if daily_group_count >= 2:
    send_meta_reply(sender_phone, "You have reached the daily limit (2) for creating new
    return {"statusCode": 200, "body": "OK"}

new_group = Group(
    group_id=uuid.uuid4(),
    owner_id=owner.user_id,
    group_name=group_name,
    status="active",
    is_active=True
)
db.add(new_group)
db.commit()

from services.audit.service import AuditService
AuditService(db).log_action(
    actor_id=str(owner.user_id),
    action="GROUP_CREATED",
    entity_type="group",
    entity_id=str(new_group.group_id),
    details={
        "message": f"New group \"{group_name}\" created via WhatsApp",
        "group_id": str(new_group.group_id)
    }
)
send_meta_reply(sender_phone, f"Success! I've created the group '{group_name}'.", confi
return {"statusCode": 200, "body": "OK"}

elif parsed_intent["intent"] == "create_campaign":
    title = parsed_intent["entities"].get("campaign_title")
    if not title:
        send_meta_reply(sender_phone, "I see you want to create a campaign! To do this, please
        return {"statusCode": 200, "body": "OK"}

from models.group import Group
from models.campaign import Campaign
import uuid
import base64
import datetime

# Check daily limit: Max 2 campaigns per day across any groups
yesterday = datetime.datetime.utcnow() - datetime.timedelta(days=1)
daily_camp_count = db.query(Campaign).join(Group).filter(
    Group.owner_id == owner.user_id,
    Campaign.created_at >= yesterday
).count()

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        if daily_camp_count >= 2:
            send_meta_reply(sender_phone, "You have reached the daily limit (2) for creating new campaigns")
            return {"statusCode": 200, "body": "OK"}

        active_groups = db.query(Group).filter(
            Group.owner_id == owner.user_id,
            Group.is_active == True
        ).all()

        if len(active_groups) == 0:
            send_meta_reply(sender_phone, "You need to create a group first before creating a campaign")
        elif len(active_groups) == 1:
            new_camp = Campaign(
                campaign_id=uuid.uuid4(),
                group_id=active_groups[0].group_id,
                title=title,
                status="active",
                is_active=True
            )
            db.add(new_camp)
            db.commit()

            from services.audit.service import AuditService
            AuditService(db).log_action(
                actor_id=str(owner.user_id),
                action="CAMPAIGN_CREATED",
                entity_type="campaign",
                entity_id=str(new_camp.campaign_id),
                details={
                    "message": f"New campaign \"{title}\" created via WhatsApp",
                    "campaign_id": str(new_camp.campaign_id),
                    "group_id": str(active_groups[0].group_id),
                    "group_slug": active_groups[0].slug,
                    "campaign_slug": new_camp.slug
                }
            )
            send_meta_reply(sender_phone, f"Success! Campaign '{title}' created under your group")
        else:
            encoded_title = base64.b64encode(title.encode('utf-8')).decode('utf-8')
            rows = []
            for g in active_groups[:10]:
                rows.append({
                    "id": f"NEW_CAMP_{g.group_id}_{encoded_title}",
                    "title": g.group_name[:24]
                })

            sections = [{
                "title": "Your Groups",
                "rows": rows
            }]

            send_meta_interactive_list(sender_phone, f"Which group should the campaign '{title}' be created under?")

        return {"statusCode": 200, "body": "OK"}

# 4. Invoke Ingestion Service Logic
ingestion_service = IngestionService(db)
result = ingestion_service.process_webhook(normalized_payload)

# Determine reply text and interactive list if configured
if result["status"] == "ignored":
    reply_text = "You have already submitted this transaction. It is currently pending review."

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        send_meta_reply(sender_phone, reply_text, config)
    elif result["status"] == "error":
        handle_unauthorized_access(sender_phone, config, db)
    elif result["status"] == "invalid":
        reply_text = "I'm sorry, I didn't understand that request. Please forward a valid M-Pesa transaction"
        send_whatsapp_main_menu(sender_phone, config, reply_text)
    else:
        parsed = result.get("parsed_data", {})
        amt = f"KES {parsed.get('amount', 0.0):.2f}" if parsed.get('amount') else "the transaction"
        name = parsed.get("sender_name") or parsed.get("provider") or "the sender"
        pending_id = result.get("pending_id")

        # Check if interactive approvals are enabled
        interactive_sent = False
        if pending_id:
            from repositories.transaction_repo import TransactionRepository
            repo = TransactionRepository(db)
            owner = repo.resolve_owner_by_phone(sender_phone)

            if owner:
                from services.settings.settings_service import SettingsService
                settings_service = SettingsService(db)
                user_settings = settings_service.get_global_settings(str(owner.user_id))

                if user_settings.automation.auto_approve_enabled and user_settings.automation.auto_approve_group_id:
                    from services.approval.service import ApprovalService
                    from models.campaign import Campaign

                    auto_campaign = db.query(Campaign).filter(Campaign.campaign_id == parse_uuid(user_settings.automation.auto_approve_campaign_id))
                    if auto_campaign:
                        try:
                            approval_service = ApprovalService(db)
                            approval_service.approve_transaction(
                                pending_txn_id=pending_id,
                                treasurer_id=str(owner.user_id),
                                group_id=user_settings.automation.auto_approve_group_id,
                                campaign_id=user_settings.automation.auto_approve_campaign_id
                            )
                            reply_text = f"Success! We received {amt} from {name}. It was automatically approved."
                            send_meta_reply(sender_phone, reply_text, config)
                            interactive_sent = True
                        except Exception as e:
                            logger.error(f"Failed to auto-approve transaction {pending_id}: {e}")

        if not interactive_sent and user_settings.automation.allow_whatsapp_approvals:
            from models.campaign import Campaign
            from models.group import Group
            active_campaigns = db.query(Campaign).join(Group).filter(
                Group.owner_id == parse_uuid(owner.user_id),
                Campaign.is_active == True
            ).order_by(Campaign.created_at.desc()).limit(10).all()

            if active_campaigns:
                rows = []
                for c in active_campaigns:
                    rows.append({
                        "id": f"APPROVE_{pending_id}_{c.campaign_id}",
                        "title": c.title[:24]
                    })

                sections = [{
                    "title": "Approve to Campaign",

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        "rows": rows
    }}

    body_text = f"Success! We received {amt} from {name}. Select a campaign below to in
    send_meta_interactive_list(sender_phone, body_text, "Approve Transaction", sections
    interactive_sent = True

    if not interactive_sent:
        reply_text = f"Success! We received {amt} from {name}. It is now awaiting for your approval in
        send_meta_reply(sender_phone, reply_text, config)

    # Acknowledge receipt immediately to Meta
    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({"status": "OK"})
    }

except Exception as e:
    logger.error(f"CRITICAL: Unexpected error in ingestion processing: {e}", exc_info=True)
    return {
        "statusCode": 500,
        "body": json.dumps({"error": "internal_server_error", "message": str(e)})
    }
finally:
    db.close()

def handle_unauthorized_access(phone: str, config, db_session):
    from models.whatsapp_blocklist import WhatsAppBlocklist

    record = db_session.query(WhatsAppBlocklist).filter(WhatsAppBlocklist.phone_number == phone).first()
    if not record:
        record = WhatsAppBlocklist(phone_number=phone, attempt_count=1, is_blocked=False)
        db_session.add(record)
        send_meta_reply(phone, f"Unauthorized Access: Your phone number is not registered as a KapuLetu treasure
    else:
        record.attempt_count += 1
        if record.attempt_count >= 3 and not record.is_blocked:
            record.is_blocked = True
            send_meta_reply(phone, "Security Alert: This phone number has been temporarily blocked from accessi
        elif not record.is_blocked:
            send_meta_reply(phone, f"Unauthorized Access: Your phone number is not registered as a KapuLetu trea

    db_session.commit()

def dispatch_whatsapp_report(db, sender_phone, campaign_id, report_text, config):
    if len(report_text) > 4000:
        from models.campaign import Campaign
        from common.utils import parse_uuid
        campaign = db.query(Campaign).filter(Campaign.campaign_id == parse_uuid(campaign_id)).first()
        if campaign:
            frontend_url = config.FRONTEND_URL.rstrip('/')
            group_slug = campaign.group.slug or campaign.group_id
            camp_slug = campaign.slug or campaign.campaign_id

            fallback_text = (
                f"*The comprehensive report for \"{campaign.title}\" is ready.*\n\n"
                f"*To view the full list of contributors and detailed metrics, please log in to your KapuLetu d
                f"{frontend_url}/treasurer/groups/{group_slug}/campaigns/{camp_slug}"
            )

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```

        send_meta_reply(sender_phone, fallback_text, config)
        return
    send_meta_reply(sender_phone, report_text, config)

def send_meta_reply(to_phone: str, message_text: str, config):
    """
    Helper function to send an outgoing text message via the Meta WhatsApp Cloud API.
    """
    if not config.META_ACCESS_TOKEN or not config.META_PHONE_NUMBER_ID:
        logger.warning("Skipping outgoing WhatsApp reply: META_ACCESS_TOKEN or META_PHONE_NUMBER_ID not configured")
        return

    url = f"https://graph.facebook.com/v19.0/{config.META_PHONE_NUMBER_ID}/messages"

    headers = {
        "Authorization": f"Bearer {config.META_ACCESS_TOKEN}",
        "Content-Type": "application/json"
    }

    payload = {
        "messaging_product": "whatsapp",
        "recipient_type": "individual",
        "to": to_phone,
        "type": "text",
        "text": {
            "preview_url": False,
            "body": message_text
        }
    }

    try:
        response = requests.post(url, headers=headers, json=payload, timeout=5)
        if response.status_code not in (200, 201):
            logger.error(f"Failed to send Meta reply. Status: {response.status_code}, Response: {response.text}")
        else:
            logger.info("Successfully sent outgoing Meta reply.")
    except Exception as e:
        logger.error(f"Error executing outgoing Meta HTTP request: {e}")

def send_whatsapp_main_menu(to_phone: str, config, prompt_text: str = None):
    text = prompt_text or "Hello! Welcome to KapuLetu. I am your automated treasury assistant. I can assist you with the following sections:"
    sections = [
        {
            "title": "Options",
            "rows": [
                {"id": "GREETING_WORKSPACE", "title": "Check your workspace", "description": "Link to KapuLetu dashboard"},
                {"id": "GREETING_APPROVALS", "title": "Pending Contributions", "description": "Review member transactions"},
                {"id": "GREETING_HELP", "title": "Help", "description": "Interactive KapuLetu guide"}
            ]
        }
    ]
    send_meta_interactive_list(to_phone, text, "Main Menu", sections, config)

def send_whatsapp_help_menu(to_phone: str, config):
    text = "Welcome to the KapuLetu Help Center. Please select a topic from the menu below to view detailed instructions:"
    sections = [
        {
            "title": "Topics",
            "rows": [
                {"id": "HELP_GROUP", "title": "Creating a Group"},
                {"id": "HELP_CAMPAIGN", "title": "Creating a Campaign"},
                {"id": "HELP_APPROVE", "title": "Approving Contributions"},
                {"id": "HELP_BOT", "title": "Using the WhatsApp Bot"},
                {"id": "HELP_SETTINGS", "title": "Settings"}
            ]
        }
    ]

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```

    ]
    }]
    send_meta_interactive_list(to_phone, text, "Help Topics", sections, config)

def send_meta_interactive_list(to_phone: str, body_text: str, button_text: str, sections: list, config):
    """
    Helper function to send an outgoing Interactive List message via the Meta WhatsApp Cloud API.
    sections format: [{"title": "Section Name", "rows": [{"id": "row_id", "title": "Row Title", "description": "Row Description"}]}]
    """
    if not config.META_ACCESS_TOKEN or not config.META_PHONE_NUMBER_ID:
        logger.warning("Skipping outgoing WhatsApp reply: META_ACCESS_TOKEN or META_PHONE_NUMBER_ID not configured")
        return

    url = f"https://graph.facebook.com/v19.0/{config.META_PHONE_NUMBER_ID}/messages"

    headers = {
        "Authorization": f"Bearer {config.META_ACCESS_TOKEN}",
        "Content-Type": "application/json"
    }

    payload = {
        "messaging_product": "whatsapp",
        "recipient_type": "individual",
        "to": to_phone,
        "type": "interactive",
        "interactive": {
            "type": "list",
            "body": {
                "text": body_text
            },
            "action": {
                "button": button_text[:20], # Max 20 chars
                "sections": sections
            }
        }
    }

    try:
        response = requests.post(url, headers=headers, json=payload, timeout=5)
        if response.status_code not in (200, 201):
            logger.error(f"Failed to send Meta interactive list. Status: {response.status_code}, Response: {response.json()}")
        else:
            logger.info("Successfully sent outgoing Meta interactive list.")
    except Exception as e:
        logger.error(f"Error executing outgoing Meta HTTP request: {e}")

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

File: services/notifications/providers/whatsapp_client.py

```
import os
import httpx
import logging

logger = logging.getLogger(__name__)

class WhatsAppClient:
    """
    A lightweight HTTP client for the WhatsApp Meta Cloud API.
    """
    def __init__(self):
        self.access_token = os.environ.get("WHATSAPP_ACCESS_TOKEN")
        self.phone_number_id = os.environ.get("WHATSAPP_PHONE_NUMBER_ID")
        self.api_version = os.environ.get("WHATSAPP_API_VERSION", "v17.0")

    def send_text_message(self, to_phone: str, message: str) -> bool:
        if not self.access_token or not self.phone_number_id:
            logger.error("WhatsApp credentials are not set. Cannot send message.")
            return False

        # Clean phone number: remove any non-digit characters.
        # Meta API requires country code without '+', e.g., '254700000000'
        clean_phone = ''.join(filter(str.isdigit, str(to_phone)))

        url = f"https://graph.facebook.com/{self.api_version}/{self.phone_number_id}/messages"
        headers = {
            "Authorization": f"Bearer {self.access_token}",
            "Content-Type": "application/json"
        }
        payload = {
            "messaging_product": "whatsapp",
            "recipient_type": "individual",
            "to": clean_phone,
            "type": "text",
            "text": {
                "preview_url": False,
                "body": message
            }
        }

        try:
            with httpx.Client() as client:
                response = client.post(url, headers=headers, json=payload, timeout=10.0)
                if response.status_code in (200, 201):
                    logger.info(f"Successfully dispatched WhatsApp message to {clean_phone}")
                    return True
                else:
                    logger.error(f"WhatsApp API error: {response.status_code} - {response.text}")
                    return False
        except Exception as e:
            logger.error(f"Failed to send WhatsApp message: {e}")
            return False
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

File: services/reporting/daily_summary.py

```
from sqlalchemy import func
from common.utils import parse_uuid
from sqlalchemy.orm import Session
from datetime import datetime

from models.campaign import Campaign
from models.transaction import Transaction

def generate_summary(db: Session, owner_id: str):
    """Calculates a real-time financial summary for the treasurer dashboard."""
    total_collected = db.query(func.sum(Transaction.amount)).filter(
        Transaction.owner_id == parse_uuid(owner_id)
    ).scalar() or 0.0

    campaign_breakdown = db.query(
        Campaign.title,
        func.sum(Transaction.amount).label("total")
    ).join(Transaction, Transaction.campaign_id == parse_uuid(Campaign.campaign_id))\
        .filter(Transaction.owner_id == parse_uuid(owner_id))\
        .group_by(Campaign.title).all()

    return {
        "total_collected": float(total_collected),
        "campaigns": [{"title": row.title, "total": float(row.total)} for row in campaign_breakdown]
    }

def generate_campaign_whatsapp_report(db: Session, campaign_id: str, manual_instructions: str = None):
    """
    Generates a Hyper-Clean, Professional WhatsApp Contribution Report.
    Prioritizes instructions set during campaign creation.
    """
    # 1. Fetch Data
    campaign = db.query(Campaign).filter(Campaign.campaign_id == parse_uuid(campaign_id)).first()
    if not campaign:
        return "ERROR: CAMPAIGN RECORD NOT FOUND."

    txns = db.query(Transaction).filter(
        Transaction.campaign_id == parse_uuid(campaign_id),
        Transaction.status == "approved"
    ).order_by(Transaction.created_at.asc()).all()

    # 2. Financial Context
    total_collected = sum(float(t.amount) for t in txns)
    target = float(campaign.target_amount) if campaign.target_amount else 0.0
    remaining_balance = max(0, target - total_collected)

    from datetime import timezone
    import zoneinfo
    now_eat = datetime.now(timezone.utc).astimezone(zoneinfo.ZoneInfo("Africa/Nairobi"))
    date_str = now_eat.strftime("%d %B %Y")

    # 3. Instruction Logic (Priority: Manual Override > Campaign DB > Default)
    instructions = manual_instructions or campaign.payment_instructions or "Pay via M-Pesa to the Treasury Number"

    # 4. Build the Hyper-Clean Structured Message
    report = f"TREASURY REPORT: {campaign.title.upper()}\n"
    report += f"STATUS REPORT AS OF {date_str.upper()}\n"
    report += f"=====\n"
    report += f"PAYMENT INSTRUCTIONS:\n"
```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```

report += f"{instructions}\n\n"

if campaign.description:
    report += f"GOAL DESCRIPTION:\n"
    report += f"{campaign.description}\n\n"

report += f"CONTRIBUTOR LIST:\n"
report += f"-----\n"
if not txns:
    report += f"(No approved records to display)\n"
else:
    counter = 1
    for t in txns:
        if t.allocations:
            # If the transaction is split, list each allocation separately
            for alloc in t.allocations:
                name = alloc.member_name or "Member"
                report += f"{counter:2}. {name:18} KES {float(alloc.allocated_amount):>8,.0f}\n"
                counter += 1
        else:
            # Standard single-member transaction
            name = t.sender_name or (f"Member {t.sender_phone[-4:]}" if t.sender_phone else "Member")
            report += f"{counter:2}. {name:18} KES {float(t.amount):>8,.0f}\n"
            counter += 1

report += f"-----\n\n"

report += f"FINANCIAL SUMMARY:\n"
report += f"TOTAL COLLECTED:    KES {total_collected:>10,.2f}\n"
if target > 0:
    report += f"CAMPAIGN TARGET:    KES {target:>10,.2f}\n"
    report += f"OUTSTANDING:        KES {remaining_balance:>10,.2f}\n"

report += f"=====\n\n"
report += f"This is an automated treasury update. "
report += f"For corrections, please contact the treasurer."

return report

```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

File: services/reporting/template_engine.py

```
from datetime import datetime
import random
from common.utils import parse_uuid
from sqlalchemy.orm import Session, joinedload
from sqlalchemy import select

from models.campaign import Campaign
from models.transaction import Transaction
from models.report_settings import CampaignReportSettings
from common.config import get_config

class TemplateEngine:
    """
    TemplateEngine: Compiles dynamic, culturally formatted WhatsApp reports.
    Uses either the Default KapuletU Standard or custom User Settings.
    """
    def __init__(self, db: Session):
        self.db = db

    def _generate_pin(self) -> str:
        return str(random.randint(1000, 9999))

    def get_or_create_settings(self, campaign_id: str) -> CampaignReportSettings:
        settings = self.db.execute(
            select(CampaignReportSettings).where(CampaignReportSettings.campaign_id == parse_uuid(campaign_id))
        ).scalars().first()

        if not settings:
            settings = CampaignReportSettings(
                campaign_id=campaign_id,
                public_access_pin=self._generate_pin()
            )
            self.db.add(settings)
            self.db.commit()
            self.db.refresh(settings)

        return settings

    def generate_whatsapp_report(self, campaign_id: str) -> str:
        # 1. Fetch Campaign and Data
        campaign = self.db.execute(select(Campaign).where(Campaign.campaign_id == parse_uuid(campaign_id))).scalars().first()
        if not campaign:
            return "ERROR: Campaign not found."

        settings = self.get_or_create_settings(campaign_id)

        txns = self.db.execute(
            select(Transaction).options(joinedload(Transaction.allocations)).where(
                Transaction.campaign_id == parse_uuid(campaign_id),
                Transaction.status == "approved"
            ).order_by(Transaction.created_at.asc())
        ).scalars().unique().all()

        # 2. Calculate Variables
        total_collected = sum(float(t.amount) for t in txns)
        target = float(campaign.target_amount) if campaign.target_amount else 0.0
        deficit = max(0, target - total_collected)

        from datetime import timezone
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

```
import zoneinfo
now_eat = datetime.now(timezone.utc).astimezone(zoneinfo.ZoneInfo("Africa/Nairobi"))
date_str = now_eat.strftime("%A %d %B, %Y")

# 3. Default KapuletU Standard Templates
default_header = (
    f"KAPULETU TREASURY UPDATE\n"
    f"*{campaign.title.upper()}*\n"
    f"As of {date_str}\n\n"
    f"Thank you for your continued support and contributions.\n\n"
    f"CONTRIBUTOR LIST:\n"
    f"-----"
)

default_footer = (
    f"-----\n"
    f"FINANCIAL SUMMARY:\n"
    f"Total Paid      = KSh. {total_collected:>10,.0f}\n"
    f"Expected Amount = KSh. {target:>10,.0f}\n"
    f"Deficit        = KSh. {deficit:>10,.0f}\n\n"
    f"{{payment_instructions}}\n"
    f"Thank you for your continued support."
)

header = settings.header_template or default_header
footer = settings.footer_template or default_footer

# Smart Tag Replacement
pay_instruct = campaign.payment_instructions if campaign.payment_instructions else ""

replacements = {
    "{{campaign_title}}": campaign.title,
    "{{current_date}}": date_str,
    "{{total_raised}}": f"{total_collected:,.0f}",
    "{{target_amount}}": f"{target:,.0f}",
    "{{deficit}}": f"{deficit:,.0f}",
    "{{payment_instructions}}": pay_instruct
}

for tag, val in replacements.items():
    header = header.replace(tag, str(val))
    footer = footer.replace(tag, str(val))

# 4. Compile Contributor List
report = header + "\n"

counter = 1
for t in txns:
    emoji = "■" if settings.use_emojis else ""
    status_txt = "(paid)" if settings.show_status_text else ""

    if t.allocations:
        for alloc in t.allocations:
            name = alloc.member_name or "Member"
            amount = float(alloc.allocated_amount)
            report += f"{counter}. {name} - {amount:,.0f} {status_txt}{emoji}\n"
            counter += 1
    else:
        name = t.sender_name or (f"Member {t.sender_phone[-4:]}" if t.sender_phone else "Member")
        amount = float(t.amount)
        report += f"{counter}. {name} - {amount:,.0f} {status_txt}{emoji}\n"
        counter += 1
```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```
# 5. Append Blank Slots (Who is next?)
if settings.blank_slots_count > 0:
    report += "\n"
    for _ in range(settings.blank_slots_count):
        report += f"{counter}.\n"
        counter += 1

report += "\n" + footer + "\n\n"

# 6. Append Public Web Link CTA
config = get_config()
frontend_url = config.FRONTEND_URL.rstrip('/')
group_id = campaign.group.slug or campaign.group_id
camp_id = campaign.slug or campaign.campaign_id
workspace_id = campaign.group.owner_id
report += (
    f"=====\n"
    f" View organized live report online:\n"
    f"{frontend_url}/report/w/{workspace_id}/g/{group_id}/c/{camp_id}\n"
    f"Access Code: {settings.public_access_pin}\n"
    f"=====
)

return report
```

KapuletU Systems - Copyright Submission Document

Proprietary & Confidential

File: ../kapuletU-frontend/src/features/finance/services/mutations.ts

```
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { toast } from "sonner";
import { apiClient } from "@lib/api-client";

interface CheckoutPayload {
  plan_id: string;
  provider: string;
  phone_number?: string;
  email?: string;
  name?: string;
  billing_cycle?: string;
  has_addons?: boolean;
}

interface CheckoutResponse {
  checkout_id: string;
  status: string;
  provider_response: Record<string, unknown>;
}

export const useInitiateCheckoutMutation = () => {
  return useMutation({
    mutationFn: async (payload: CheckoutPayload): Promise<CheckoutResponse> => {
      const response = await apiClient.post<CheckoutResponse>("/finance/checkout", payload);
      return response.data;
    },
  });
};

export const useActivateTrialMutation = () => {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: async (): Promise<{ message: string; status: string }> => {
      const response = await apiClient.post<{ message: string; status: string }>(
        "/finance/activate-trial",
      );
      return response.data;
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["my-subscription"] });
      queryClient.invalidateQueries({ queryKey: ["me"] });
    },
  });
};

export const useExportReceiptPdfMutation = () => {
  return useMutation({
    mutationFn: async (paymentId: string) => {
      const response = await apiClient.get(`/finance/receipt/${paymentId}`, {
        responseType: "blob",
      });
      return { data: response.data, paymentId };
    },
    onError: (error) => {
      toast.error(error instanceof Error ? error.message : "Failed to download receipt.");
    },
    onSuccess: ({ data, paymentId }) => {
      const url = window.URL.createObjectURL(new Blob([data]));
      const link = document.createElement("a");

```

Kapuletu Systems - Copyright Submission Document

Proprietary & Confidential

```
        link.href = url;
        link.setAttribute("download", `receipt-${paymentId}.pdf`);
        document.body.appendChild(link);
        link.click();
        link.remove();
        window.URL.revokeObjectURL(url);
    },
    });
};
```